Ryan Maxin, Isha Chaudhry, Nandish Patel

PLAN OF ATTACK

PROJECT: WATOPOLY

Team Success Mantra: The only way to succeed in this project (including getting full + bonus points), think of it like it is YOUR project and the other two team members are helping in giving hints.

Questions:

1. *After reading this subsection, would the Observer Pattern be a good pattern to use when implementing a gameboard? Why or why not?*

Before agreeing that the Observer Pattern would be a good pattern to use when implementing the gameboard, we initially had decided not to; however, after deciding on using MVC (Model View Controller), we believe the observer pattern would be an excellent pattern to use. The observer pattern has many benefits when working with the MVC architecture since it helps reduce coupling, where the changes can occur in the components, but they don't need to affect one another, making the code easier to modify and increasing flexibility.

The Observer Pattern helps with flexibility since it allows new views to be added without impacting the model. New views can include a console interface, a graphical UI or a mobile app interface. Not only this, but we also agree that using MVC and Observer Pattern separates the logic and UI (user interface). Having this separation increases flexibility and is more effective in not affecting the watopoly's game logic. This can help significantly when debugging, testing, adding more changes/features to a specific component and overall, not requiring the changes to be implemented or affecting other parts.

2. *Suppose that we wanted to model SLC and Needles Hall more closely to Chance and Community Chest cards. Is there a suitable design pattern you could use? How would you use it?*

The Community Chest and Chance cards offer a challenge to implement in that each card has different behaviour, as opposed to the predictable outcomes from Watapoly's SLC and Needles Hall. Obviously, the Chance and Community Chest cards would have to be hard-coded somewhere, but ideally, we do not want to hardcode it into our gameboard logic. I don't see a specific design pattern, at least from the ones we've seen in class, that would help with this problem. This would imply we would maybe want to use a design pattern such as the action pattern/command pattern (which was not covered in class). Using this, we could have Needles Hall own a "Needles Hall" class, and SLC own an "SLC" class. Both of these classes would have a pure virtual method use(). The children of Needles Hall and SLC would be wrapper classes which would each have an overridden use() function. This function, which would be called by its parent's use(), would execute the desired effect of the card. Each of these wrapper classes could be instantiated inside of an array in Needles Hall or SLC so that the gameboard can simply assign a NeedlesHall* or SLC* to a random index in this array (using <random>). Finally, the gameboard can call ->use() to get a random effect. The benefit is that to change the cards, such as if you had a different set of Chance cards or if you wanted fewer or more cards in play, you only have to change the NeedlesHall.cc or SLC.cc.

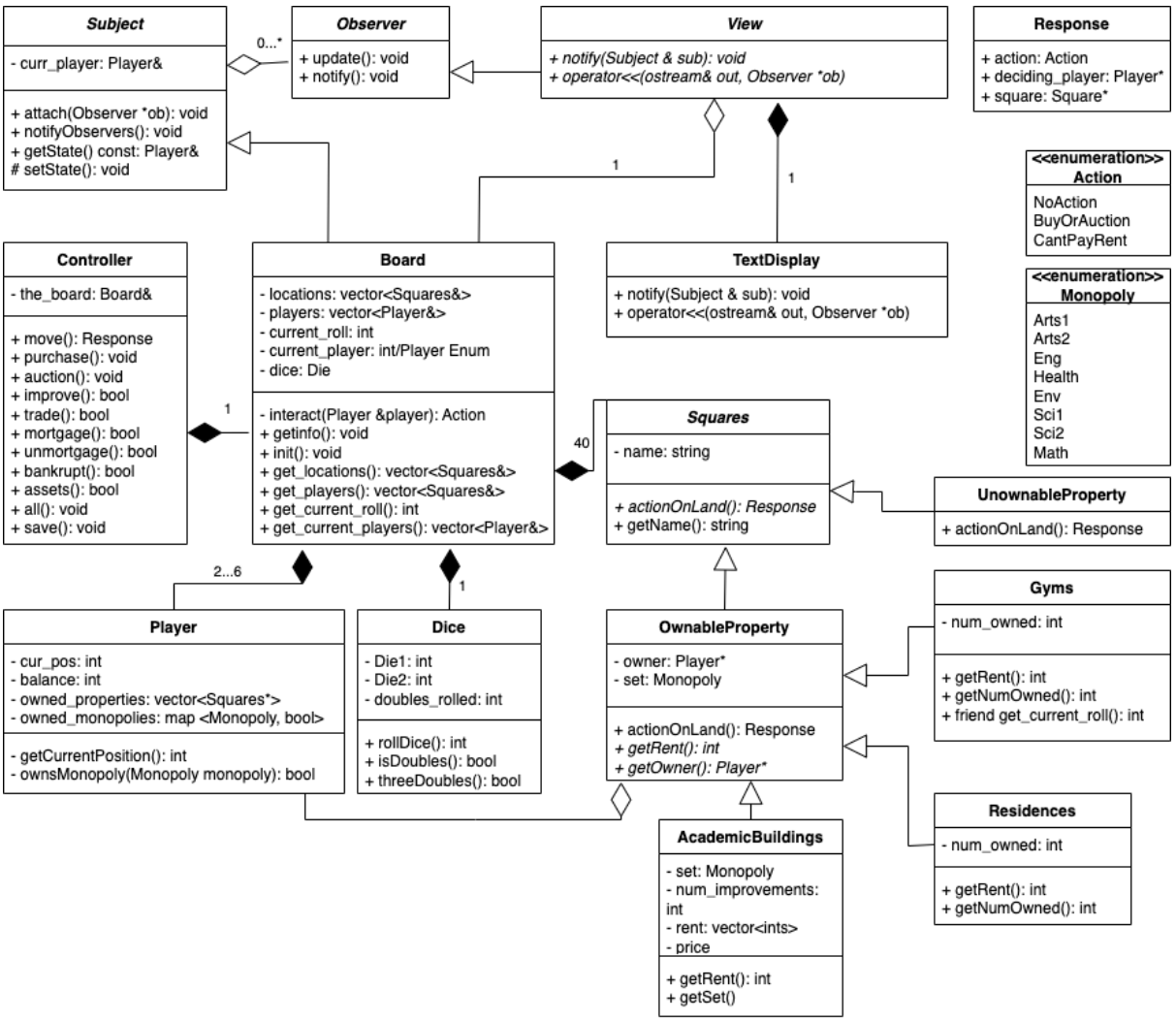
3. *Is the Decorator Pattern a good pattern to use when implementing Improvements? Why or why not?*

Initially, our team thought the decorator pattern would be a good idea to use for modelling improvements on the board. While adding complexity to the program, the decorator pattern shines in the fact that it allows features to be added dynamically to a class. Intuitively, this makes sense with improvements because instead of spending a tedious amount of time to hardcode the tuition price after each improvement, you could simply take the tuition from all previous improvements and the base property and add the tuition that the current improvement adds.

As we discovered, the issue with the decorator pattern was that finding the tuition the current improvement adds is not really feasible. Initially, it seems like the tuition is multiplied by 3, but you can soon realize this isn't the case. After doing some research, it turns out there is actually no pattern at all for the scaling of Monopoly improvements. If there is no pattern, then the tuition prices for each improvement must be hardcoded, meaning we can't find the rent price dynamically. This means all of the benefits of the decorator pattern are gone, but the added complexity is still present. For this reason, we decided not to use the decorator pattern.

Breakdown:

Ryan Maxin, Isha Chaudhry, Nandish Patel



Our design follows MVC, with our gameboard and all of the classes it owns as part of the model/ the logic of the game.

We want to create the program's central control flow, which means working with how the user interacts with the program, so we want to start with the Controller class.

Next, we want to work on the actual logic of the game, fleshing out the methods we designed in the controller and creating vital classes like Player and Board. Implementing these classes first helps to increase functionality further by adding more features. Next, we can move on to the properties on the board itself and work on how the player can interact with these properties. After all of our classes are finished, we can begin testing our program (using print statements to understand what is going on at key locations in our program) to ensure we have basic functionality.

After this, we can create the view, which, initially, for us will be the text display. We believe it is best to implement the text display after implementing the basic functionality of the game logic as part of the initial stages. This will allow us actually to see our game in action, and we can start dealing with edge cases or tricky bugs at this point.

Lastly, if all goes well, we can add additional features (like a graphical display) or whatever interests us.

Ryan Maxin, Isha Chaudhry, Nandish Patel

Plan:

We have the following meetings scheduled:
April 1 (3 pm - 9 pm):
Goals:

- *General*: Set up repository, discuss style guide, make Makefile, restrictions (need two checks to git merge), update UML as needed. Work together on board.h, player.h, controller.h, squares.h (and children). Discuss main functionality. Create board.cc together (liveshare). Set up
- *Nandish*: make pseudocode for player.cc, start implementation
- *Ryan*: make pseudocode for squares.cc (and children), start implementation
- *Isha*: make pseudocode for controller.cc, start implementation

April 2 (10 am - 4 pm):
Goals: Come up with the pseudo code for the Model part in MVC

- *General*: Fix April 1 bugs, update UML as needed, work together on board.cc
- *Nandish*: work more on squares.cc (and children)
- *Ryan*: Finish most of player.cc (and children)
- *Isha*: Finish most of controller.cc (and children)

April 5 (5 pm - 8 pm):
Goals: Finish implementation

- *General*: Fix April 2 bugs, update UML as needed, tackle problem areas
- *Nandish*: Work on View.cc View.h
- *Ryan*: finish work on squares.cc (and children)
- *Isha*: Work on View.cc, View.h

April 6 (5 pm - 8 pm):
Goals: Breaking the code

- *General*: Fix April 5 bugs, Finish View.cc, test program, work on edge cases

April 7 (5 pm - 10 pm):
Goals: Submitting it

- *General*: Final changes to project. Clean it up. Extra features (if time permits)